УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность 5-04-0612-02

Учебный предмет Инструментальное программное обеспечение

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____М.М.Федоськова
_____

ЛАБОРАТОРНАЯ РАБОТА № 13

СОЗДАНИЕ ПРОСТЕЙШИХ ПРОГРАММ С ПРИМЕНЕНИЕМ НАСЛЕДОВАНИЯ И ПОЛИМОРФИЗМА

ЛАБОРАТОРНАЯ РАБОТА № 13.2

СОЗДАНИЕ ПРОСТЕЙШИХ ПРОГРАММ С ПРИМЕНЕНИЕМ ПОЛИМОРФИЗМА

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал преподаватель
Денисовец Д.А.

2026

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № ____от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Создание простейших программ с применением полиморфизма

2 Методическое и материальное обеспечение

2.1 Персональный компьютер
2.2 Методические рекомендации по выполнению лабораторной работы
2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями
3.2 Выполнить индивидуальное задание
3.3 Оформить отчет
3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

4.1 Переопределение функционала базового класса

Полиморфизм — это возможность объектов разных классов реагировать на одинаковые вызовы методов по-разному. В Python полиморфизм реализуется через переопределение методов.

Класс Employee полностью перенимает функционал класса Person:

```
class Person:
    def __init__(self, name):
        self.__name = name   # имя человека
    @property
    def name(self):
        return self.__name
    def display_info(self):
        print(f"Name: {self.__name} ")
class Employee(Person):
    def work(self):
        print(f"{self.name} works")
```

Но что, если нужно что-то изменить из этого функционала? Например, добавить работнику через конструктора, новый атрибут, который будет хранить компанию, где он работает или изменить реализацию метода display_info. Python позволяет переопределить функционал базового класса.

Например, изменим классы следующим образом:

```
class Person:
    def __init__(self, name):
        self.__name = name   # имя человека
    @property
    def name(self):
```

```
        return self.__name
    def display_info(self):
        print(f"Name: {self.__name}")
class Employee(Person):
    def __init__(self, name, company):
        super().__init__(name)
        self.company = company
    def display_info(self):
        super().display_info()
        print(f"Company: {self.company}")
    def work(self):
        print(f"{self.name} works")
tom = Employee("Tom", "Microsoft")
tom.display_info()  # Name: Tom
                    # Company: Microsoft
```

Здесь в классе Employee добавляется новый атрибут - self.company, который хранит компания работника. Соответственно метод __init__() принимает три параметра: второй для установки имени и третий для установки компании. Но если в базом классе определен конструктор с помощью метода __init__, и мы хотим в производном классе изменить логику конструктора, то в конструкторе производного класса мы должны вызвать конструктор базового класса. То есть в конструкторе Employee надо вызвать конструктор класса Person.

Для обращения к базовому классу используется выражение super(). Так, в конструкторе Employee выполняется вызов:

```
super().__init__(name)
```

Это выражение будет представлять вызов конструктора класса Person, в который передается имя работника. И это логично. Ведь имя работника устанавливается именно в конструкторе класса Person. В самом конструкторе Employee лишь устанавливаем свойство company.

Кроме того, в классе Employee переопределяется метод display_info() - в него добавляется вывод компании работника. Причем мы могли определить этот метод следующим образом:

```
def display_info(self):
    print(f"Name: {self.name}")
    print(f"Company: {self.company}")
```

Но тогда строка вывода имени повторяла бы код из класса Person. Если эта часть кода совпадает с методом из класса Person, то нет смысла повторяться, поэтому опять же с помощью выражения super() обращаемся к реализации метода display_info в классе Person:

```
def display_info(self):
    super().display_info()     # обращение к методу display_info в классе Person
    print(f"Company: {self.company}")
```

Затем мы можем вызвать вызвать конструктор Employee для создания объекта этого класса и вызвать метод display_info:

```
tom = Employee("Tom", "Microsoft")
```

```
tom.display_info()
```

Консольный вывод программы:

```
Name: Tom
Company: Microsoft
```

4.2 Проверка типа объекта

При работе с объектами бывает необходимо в зависимости от их типа выполнить те или иные операции. И с помощью встроенной функции isinstance() мы можем проверить тип объекта. Эта функция принимает два параметра:

```
isinstance(object, type)
```

Первый параметр представляет объект, а второй - тип, на принадлежность к которому выполняется проверка. Если объект представляет указанный тип, то функция возвращает True. Например, возьмем следующую иерархию классов Person-Employee/Student:

```
class Person:
    def __init__(self, name):
        self.__name = name   # имя человека
    @property
    def name(self):
        return self.__name
    def do_nothing(self):
        print(f"{self.name} does nothing")
#  класс работника
class Employee(Person):
    def work(self):
        print(f"{self.name} works")
#  класс студента
class Student(Person):
    def study(self):
        print(f"{self.name} studies")
def act(person):
    if isinstance(person, Student):
        person.study()
    elif isinstance(person, Employee):
        person.work()
    elif isinstance(person, Person):
        person.do_nothing()
tom = Employee("Tom")
bob = Student("Bob")
sam = Person("Sam")
act(tom)    # Tom works
act(bob)    # Bob studies
act(sam)    # Sam does nothing
```

Здесь класс Employee определяет метод work(), а класс Student - метод study.

Здесь также определена функция act, которая проверяет с помощью функции isinstance, представляет ли параметр person определнный тип, и зависимости от результатов проверки обращается к определенному методу объекта.

4.3 Пример решения задачи на языке программирования Python

Задача. Разработайте иерархию классов для моделирования различных видов животных с использованием принципа полиморфизма. Создайте базовый класс Animal, содержащий метод make_sound(), который должен быть переопределён в дочерних классах. Реализуйте три класса-наследника: Dog, Cat и Bird. В каждом из них переопределите метод make_sound() так, чтобы он возвращал характерный звук животного. Создайте список объектов различных классов (Dog, Cat, Bird) и организуйте их обработку в едином цикле, вызывая метод make_sound() для каждого объекта. Обратите внимание, что один и тот же метод должен выполнять разные действия в зависимости от типа объекта (полиморфизм). Выведите результаты работы программы на экран.

Решение задачи:

```
class Animal:
    def make_sound(self):
        pass
class Dog(Animal):
    def make_sound(self):
        return "Woof!"
class Cat(Animal):
    def make_sound(self):
        return "Meow!"
class Bird(Animal):
    def make_sound(self):
        return "Tweet!"
# Полиморфное использование
animals = [Dog(), Cat(), Bird()]
for animal in animals:
    print(animal.make_sound())
# Вывод:
# Woof!
# Meow!
# Tweet!
```

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Создайте базовый класс Animal с методом sound(). В классах Cat и Dog переопределите данный метод.

2 Создайте базовый класс Shape с методом area(). В классах Square и Triangle реализуйте вычисление площади.

3 Создайте базовый класс Transport с методом move(). В классах Car и Bicycle переопределите способ передвижения.

4 Создайте базовый класс Person с методом hello(). В классах Human, Robot и Alien реализуйте различные приветствия.

5 Создайте базовый класс Employee с методом salary(). В классах Manager и Programmer реализуйте разные способы расчета заработной платы.

6 Создайте базовый класс Shape с методом perimeter(). В классах Circle и Rectangle переопределите данный метод.

7 Создайте базовый класс Document с методом print_info(). В классах Book и Magazine реализуйте различный вывод информации.

8 Создайте базовый класс File с методом info(). В классах AudioFile и ImageFile переопределите данный метод.

9 Создайте базовый класс Vehicle с методом max_passengers(). В классах Bus и Train задайте различные значения вместимости.

10 Создайте базовый класс Animal с методом attack(). В классах Lion и Tiger реализуйте различные варианты атаки.

11 Создайте базовый класс Aircraft с методом fly(). В классах Airplane, Helicopter и Rocket переопределите данный метод.

12 Создайте базовый класс Payment с методом pay(). В классах CashPayment и CardPayment реализуйте различные способы оплаты.

13 Создайте базовый класс Notification с методом send(). В классах EmailNotification и SMSNotification реализуйте различные способы отправки сообщений.

14 Создайте базовый класс Payment с методом pay(). В классах CashPayment и CardPayment реализуйте различные способы оплаты.

15 Создайте базовый класс Notification с методом send(). В классах EmailNotification и SMSNotification реализуйте различные способы отправки уведомлений.

5.2 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Создайте два класса разных типов, например Cat и Dog, и реализуйте в каждом метод sound(), который выводит характерный звук животного. Создайте список объектов этих классов и выполните полиморфный вызов метода sound() для каждого объекта.

2 Создайте классы Car и Bicycle, в которых реализуйте метод move(), возвращающий способ передвижения данного транспорта. Создайте список объектов и вызовите метод move() для каждого из них.

3 Создайте классы Circle и Rectangle, каждый должен содержать метод area(), возвращающий площадь фигуры. Организуйте полиморфный вызов метода area() для разных фигур.

4 Создайте классы Teacher и Student, реализуйте метод role(), возвращающий строку, описывающую роль человека. В цикле вызовите метод role() для всех объектов.

5 Создайте классы Eagle и Penguin, реализуйте метод fly(), который выводит разную информацию (например, "летаю" и "не летаю"). Выполните полиморфный вызов метода.

6 Создайте классы Rock и Jazz, реализующие метод play(), который выводит характер звучания музыки. Выполните вызов метода play() для объектов обоих классов.

7 Создайте классы Bus и Taxi с методом transport_passenger(), описывающим процесс перевозки пассажиров.

8 Создайте классы Lion и Elephant, в каждом реализуйте метод food(), который возвращает тип пищи животного. В цикле вызовите метод для всех животных.

9 Создайте классы Winter и Summer с методом activity(), возвращающим характерные виды деятельности для данного времени года.

10 Создайте классы Firefighter и Policeman, реализующие метод duty(), который возвращает, какую работу выполняет специалист. Проверьте полиморфизм в действии.

11 Создайте классы Printer и Scanner, реализующие метод action(), описывающий работу устройства. Выполните вызов метода для списка объектов.

12 Создайте классы Plane и Helicopter, реализующие метод flight_type(), который выводит тип полёта. Выполните вызов метода для списка объектов.

13 Создайте классы Painter и Sculptor, реализуйте метод create_art(), который выводит описание художественного процесса. Проверьте работу полиморфизма на списке объектов.

14 Создайте классы EmailNotification и SMSNotification, реализуйте метод send(), который возвращает информацию о способе отправки уведомления. Организуйте полиморфный вызов метода.

15 Создайте классы Guitar и Piano, реализуйте метод play(), который выводит характер звучания музыкального инструмента. Проверьте работу полиморфизма на списке объектов.

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

6.1 Тема лабораторной работы

6.2 Цель лабораторной работы

6.3 Индивидуальное задание (текст программы и скриншоты выполнения)

7 Контрольные вопросы

7.1 Дайте определение понятию «Полиморфизм» в языке программирования Python?

7.2 Опишите разницу между полиморфизмом и наследованием в языке программирования Python?

7.3 Охарактеризуйте переопределение метода в языке программирования Python?

Список используемых источников

1 Постолит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.

2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.

3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.